

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 6 (Due 10/20)

1. Branch Prediction

Consider the following MIPS code with three branches at addresses 0x104, 0x10c and 0x130:

```

0x100      LOOP:      lw      $s1, 0($s2)
0x104                      beq      $s1, $s0, DECR
0x108                      addi     $s1, $s1, 1
0x10c                      beq      $s0, $s0, STORE

                DECR:
0x120                      addi     $s1, $s1, -1

                STORE:
0x124                      sw      $s1, 0($s2)
0x128                      addi     $s2, $s2, 4
0x12c                      sub      $s4, $s3, $s2
0x130                      bne      $s4, $s0, LOOP

```

Assume the program executes and produces the sequence of branches in the table below. The processor's branch predictor employs a branch history table (BHT) with 8 entries, each containing a 1-bit prediction (1=taken). BHT entries are indexed using bits 4 to 2 of the PC of the branch instructions. Assume that all BHT entries are initially set to 0=not taken.

(a) Fill in the BHT entries and predictions below.

Instruction PC	Taken (T) or not (N)?	BHT index (binary)	BHT entry value before the prediction	Prediction	BHT entry value after the branch executes
0x104	T	001	0	N	1
0x130	T	100	0	N	1
0x104	N	001	1	T	0
0x10c	T	011	0	N	1
0x130	T	100	1	T	1
0x104	T	001	0	N	1
0x130	T	100	1	T	1
0x104	N	001	1	T	0
0x10c	T	011	1	T	1
0x130	T	100	1	T	1

(b) What is the branch misprediction rate?

Answer:

6 mispredictions out of 10 = 60% misprediction rate

2. Control Hazards

Assume the five-stage pipeline with full forwarding and bypassing. Assume that branch decisions are made at the ID stage of the pipeline and that **data forwarding** from the EX stage is available to the branch decision circuit at the ID stage. Assume no branch delay slot. Use a predict-not-taken policy: until the branch decision is resolved, instructions on the branch-not-taken path continue to be fetched and are then **flushed** if the branch is found to be taken. Consider the following two code segments:

Code Segment A	Code Segment B
L1: lw \$t1, 0(\$t2)	L1: add \$t1, \$s0, \$s1
L2: lw \$t3, 0(\$s2)	L2: add \$t1, \$s0, \$t1
L3: beq \$t1, \$t3, L1	L3: beq \$t1, \$0, L1
L4: sub \$t4, \$t2, \$s1	

- (a) For code segment A, complete the pipeline timing diagram until L4 is executed. Use “*” to indicate an instruction stalling (e.g., "D*" implies stalling in decode) and “=” to indicate an instruction getting flushed. Assume the branch is taken the first time and not taken the next time.

Answer:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L1	F	D	X	M	W												
L2		F	D	X	M	W											
L3			F	D*	D*	D	X	M	W								
L4				F*	F*	F	=										
L1							F	D	X	M	W						
L2								F	D	X	M	W					
L3									F	D*	D*	D	X	M	W		
L4										F*	F*	F	D	X	M	W	

- (b) For code segment B, identify all the data dependences. For each one, specify if it leads to a data hazard, if forwarding is needed and if any stall cycles are needed.

Answer:

RAW: L1-L2 (\$t1, hazard, EX forwarding, no stall)

RAW: L2-L3 (\$t1, hazard, EX forwarding, one stall cycle)

WAR: L3-L1 (\$t1, no hazard, no stall)

Optional:

WAW: L1-L2 (\$t1), WAW: L2-L1 (\$t1), WAR: L3-L2 (\$t1), WAW: L1-L1 (\$t1), WAR: L2-L2 (\$t1), WAW: L2-L2 (\$t1)

3. Consider a direct-mapped cache for a 32-bit MIPS processor as we've encountered in our class. It has enough storage to cover 64 blocks, with 32 bytes per block. The metadata will consist of a valid bit and the tag bits for each block.
- (a) How many total bits of **metadata** will this cache have?

Answer:

$\log_2(32)$ gives 5 bits for offset byte, $\log_2(64)$ gives 6 bits for index, which leaves $32-6-5=21$ bits for the tag. 21 tag bits + 1 valid bit = total of 22 metadata bits. 22 bits per block * 64 blocks = 1,408 bits. Index bits are not stored, as they're implicit given the cache index (address with index 6'h10 is stored in row 8).

- (b) What % of total storage will metadata consume?

Answer: 1408 bits is 176 bytes. Data will hold $64*32 = 2048$ bytes. $176/2048$ is 8.6%

4. Consider the following trace of memory read references of **8-bit memory addresses** (given in decimal). Memory is byte-addressable.
- 21, 166, 201, 143, 61, 166, 62, 133, 111, 143, 144, 61**

- (a) Assume a **direct-mapped** cache with 8-byte blocks and a total capacity of 8 blocks. For each of these references, specify the **set index** (in binary) and **tag** (in binary) of the access. Also list if each reference is a **hit or miss**, assuming the cache is initially empty.

Answer: 3-bit (2:0) byte offset, 3-bit index (5:3) and $8-3-3 = 2$ -bit tag (7:6).

Trace	Binary address	Tag	Index	Hit/Miss
21	00010101	00	010	
166	10100110	10	100	
201	11001001	11	001	
143	10001111	10	001	
61	00111101	00	111	
166	10100110	10	100	H
62	00111110	00	111	H

133	10000101	10	000	
111	01101111	01	101	
143	10001111	10	001	H
144	10010000	10	010	
61	00111101	00	111	H